

Credit Card Payment System

Team review and demo guide

Frontend	React, Vite, Tailwind CSS
Backend	Django REST Framework and FastAPI
Database	MySQL demo dump included; screenshots use isolated SQLite
Payment mode	Simulated only; no real gateway is connected
Captured	2026-10-05

This review documents the running application using synthetic showcase records. The screenshots do not use the local MySQL records or real payment-card data.

Local showcase accounts: **demo-user** and **demo-admin**. Their passwords are intentionally omitted from this report; they are seeded only in the ignored local SQLite showcase database.

Architecture at a glance

The application separates the customer experience, core account and ledger APIs, and payment simulation. Django is the source of truth for users, card records, and transactions. FastAPI coordinates a simulated payment and calls Django's protected internal endpoints.

Caller	Protocol	Service / responsibility
React + Tailwind	JWT	Django REST: auth, cards, history, admin reporting
React + Tailwind	JWT	FastAPI: validate request and simulate payment
FastAPI	Internal secret	Django internal API: ownership check and transaction state
Django ORM	Database	MySQL in normal configuration; SQLite for screenshots

What happens during a payment

1. The signed-in React client sends the user ID, saved card ID, amount, and JWT to FastAPI. 2. FastAPI verifies the JWT and user identity. 3. Django verifies card ownership and creates a PENDING transaction. 4. FastAPI simulates the outcome and asks Django to finalize it. Django permits only a one-way PENDING-to-final transition.

Security and data handling

Django hashes passwords. Protected APIs require JWT authentication. Card entry validates the number but stores only a masked value and last four digits; CVV is write-only and discarded. FastAPI receives a saved-card ID, never the PAN or CVV. No payment gateway is used.

Review meeting walkthrough

Suggested opening: *This project demonstrates a full-stack payment workflow without moving real money. React handles the customer and admin screens, Django owns authenticated records, and FastAPI simulates payment outcomes.*

Demo sequence

1	Sign in as demo-user and point out the overview metrics and recent activity.
2	Open Cards and show the masked saved card ending in 4242.
3	Submit INR 825.00 for a simulated SUCCESS response.
4	Submit INR 100.13 for the deterministic simulated FAILED response.
5	Open Activity to show both new ledger entries, then review Admin reporting and API docs.

Verification

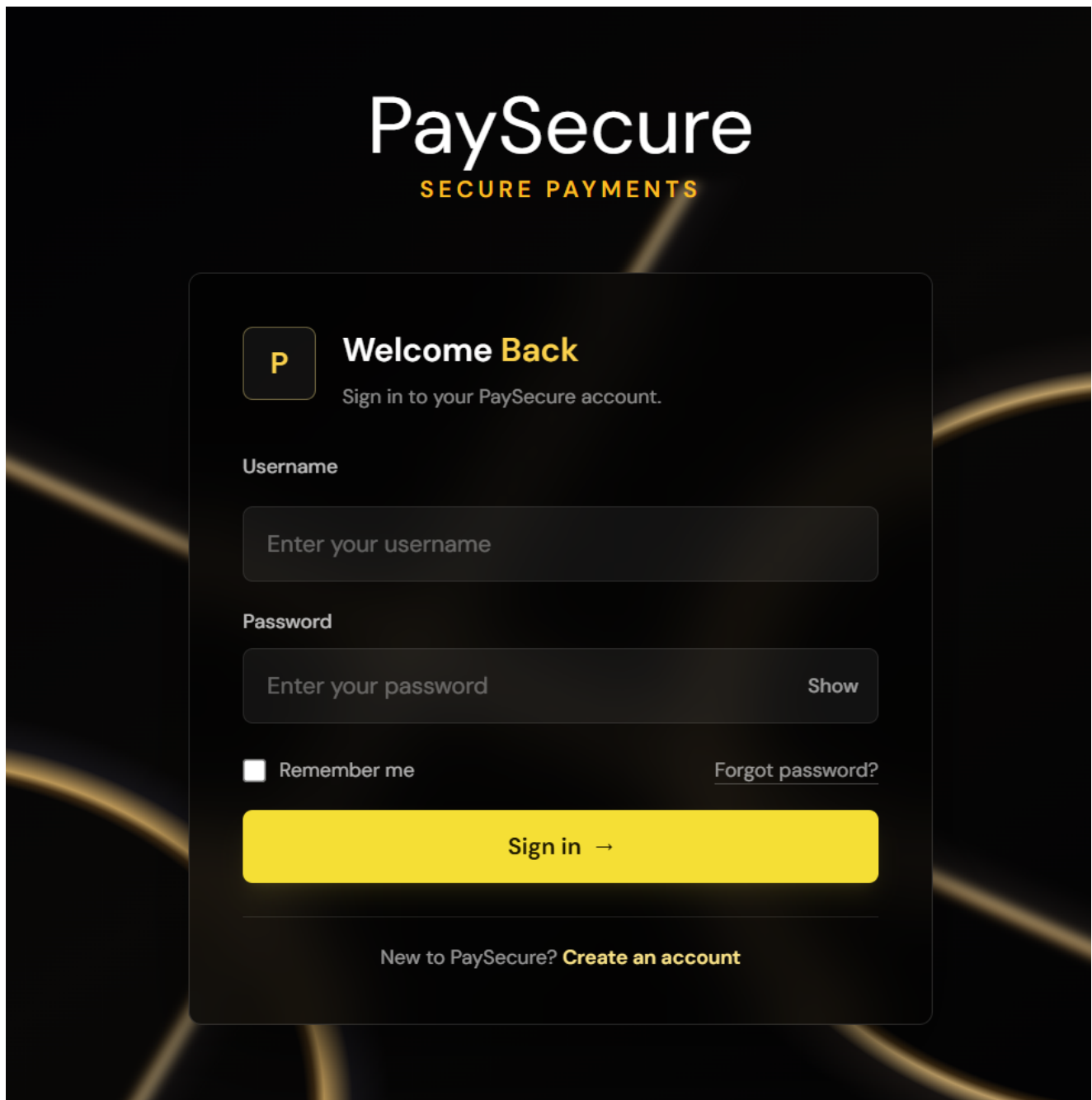
Django	10 tests passed
FastAPI	8 tests passed
Frontend	2 tests passed; production build succeeded
Docker	Not verified in this environment; Docker CLI unavailable

Scope and caveats

Payment outcomes are deterministic demo behavior, not a production processor. The included MySQL dump uses schema-only DDL plus synthetic showcase rows and creates a separate `credit_card_submission_demo` database; existing MySQL rows were not exported or modified. Screenshots use isolated SQLite. Import requires a MySQL administrator because the app account cannot create databases. This guide is not the original assignment PDF.

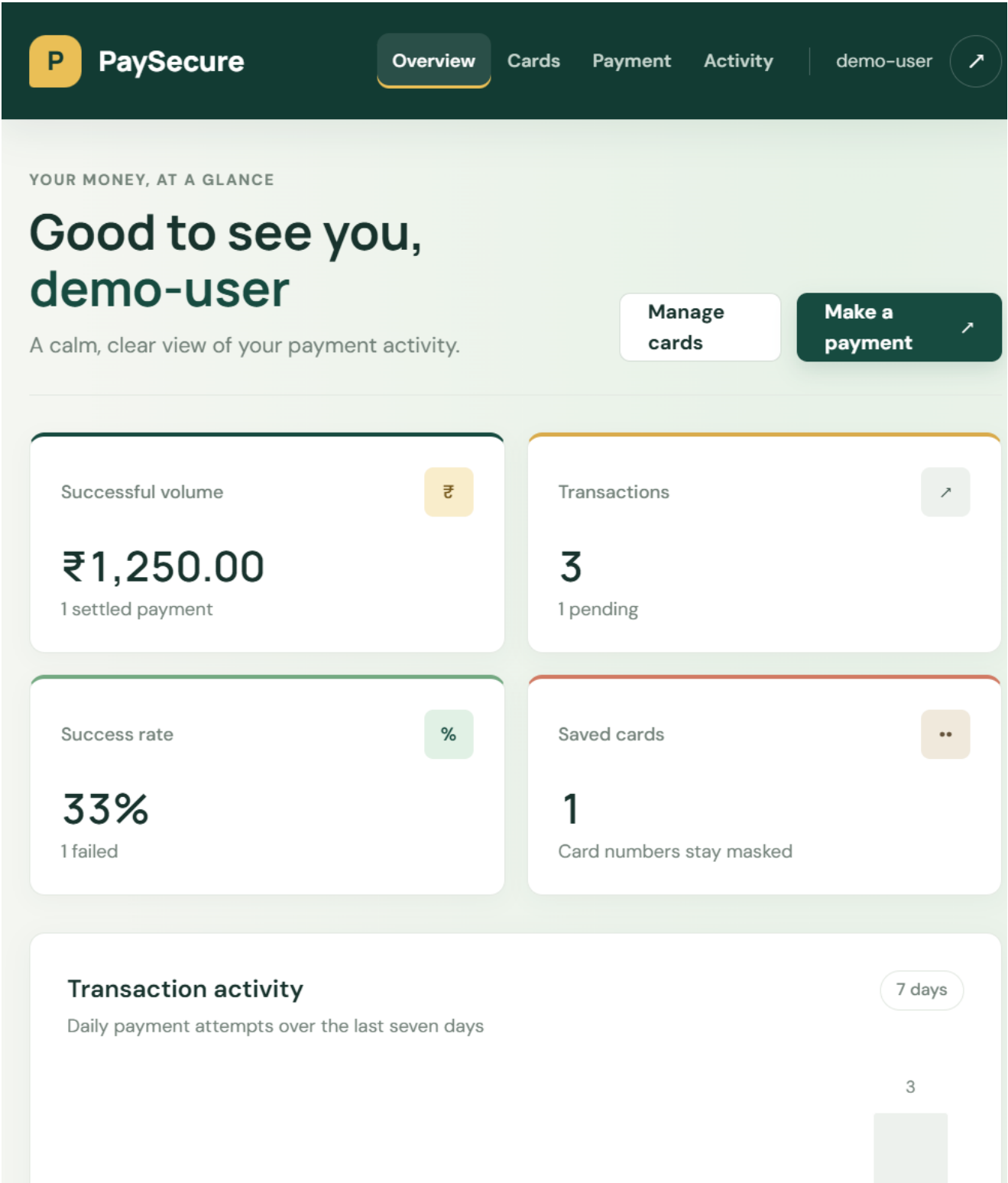
Complete PNG captures are included in the screenshots folder. The PDF uses a preview crop for layout; open the PNG for the full-page image.

Sign in

The image shows a sign-in interface for 'PaySecure'. At the top, the logo 'PaySecure' is displayed in white, with 'SECURE PAYMENTS' in yellow below it. The background is dark with abstract yellow light streaks. A central dark grey box contains the sign-in form. Inside this box, there's a yellow square icon with a white 'P' on the left. To its right, the text 'Welcome Back' is in white, with 'Back' in yellow. Below this, a subtitle says 'Sign in to your PaySecure account.' in white. The form has two input fields: 'Username' and 'Password'. The 'Username' field has a placeholder 'Enter your username'. The 'Password' field has a placeholder 'Enter your password' and a 'Show' button on the right. Below the password field, there's a checkbox labeled 'Remember me' and a link 'Forgot password?'. A large yellow button with the text 'Sign in →' is positioned below these. At the bottom of the form box, there's a link 'New to PaySecure? Create an account'.

JWT sign-in screen for the local demo account.

Customer overview



Payment totals, recent activity, and masked saved-card summary.

Cards

P

PaySecure

Overview

Cards

Payment

Activity

demo-user

PAYMENT METHODS

Your cards

Keep your payment methods together and ready to use.

1 saved

NEW METHOD

01

Add a card

PAYSECURE

..... 0000

CARDHOLDER NAME

12/30

CREDIT

Card type

CREDIT

Name on card

e.g. Alex Morgan

Card number

0000 0000 0000 0000

Card entry and saved-card views; only the masked number is persisted.

Simulated payment success

P

PaySecure

Overview

Cards

Payment

Activity

demo-user

SECURE CHECKOUT

Make a payment

A simple, secure way to move money.

Protected payment

PAYMENT DETAILS

01

Where should we send it?

✓

Payment complete

Payment successful (simulated).

Reference PAY-20261005043643-4077FD1E

Pay with

CREDIT . *****4242

Amount

825.00

INR

Payment method

Processing

CREDIT ending 4242

Instant

Pay securely

End-to-end success response from the simulated payment service.

Simulated payment failure

P

PaySecure

Overview

Cards

Payment

Activity

demo-user

SECURE CHECKOUT

Make a payment

A simple, secure way to move money.

Protected payment

PAYMENT DETAILS01

Where should we send it?

! Payment not completed
Payment failed (simulated).
Reference PAY-20261005043702-B2E59174

Pay with

CREDIT . *****4242

Amount

100.13INR

Payment method

Processing

CREDIT ending 4242

Instant

Pay securely

Deterministic failure response from the simulated payment service.

Credit Card Payment System | Local showcase

8

Transaction history

P

PaySecure

Overview

Cards

Payment

Activity

demo-user

YOUR PAYMENT RECORD

Activity

5 transactions shown

Every payment, clearly accounted for.

Filter activity

Status

From

To

All statuses

mm/dd/yyyy

mm/dd/yyyy

Minimum

Maximum

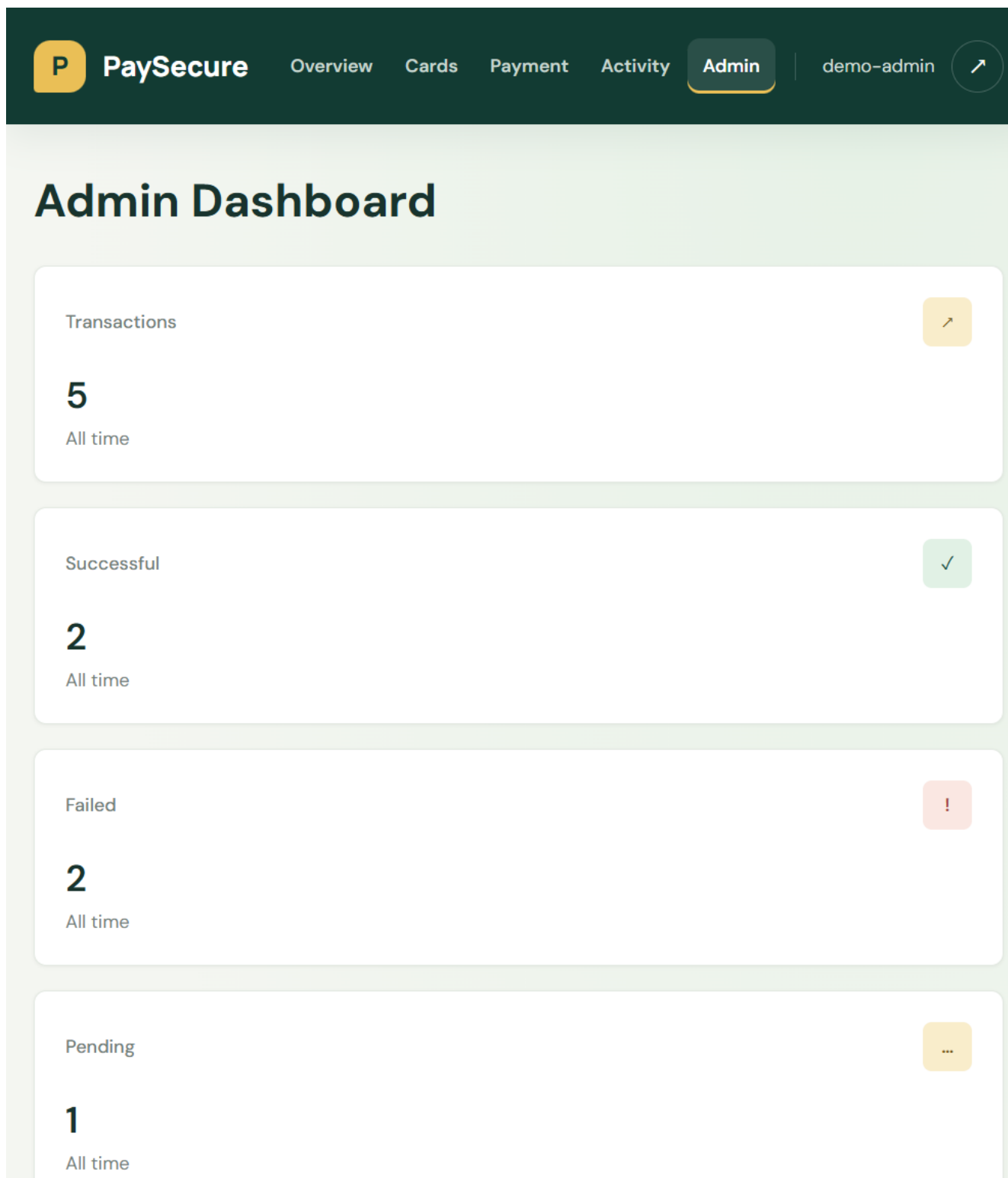
₹ 0.00

₹ 0.00

REFERENCE	AMOUNT	STATUS	PAYMENT CARD	DATE & TIME
20261005043702-B2E59174	₹100.13	FAILED	*****4242	10/5/2026, 10:07:03 AM
20261005043643-4077FD1E	₹825.00	SUCCESS	*****4242	10/5/2026, 10:06:45 AM
10-PENDING-003	₹780.00	PENDING	*****4242	10/5/2026, 10:04:04 AM
10-FAILED-002	₹340.13	FAILED	*****4242	10/5/2026, 10:04:04 AM
10-SETTLED-001	₹1,250.00	SUCCESS	*****4242	10/5/2026, 10:04:04 AM

Filterable history showing success, failure, and pending states.

Admin reporting



Staff-only transaction totals and daily reporting.

Django administration

Django administration

WELCOME, **DEMO-ADMIN**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#) 

Site administration

AUDIT

Admin logs

[+ Add](#) [✎ Change](#)

AUTHENTICATION AND AUTHORIZATION

Groups

[+ Add](#) [✎ Change](#)

Users

[+ Add](#) [✎ Change](#)

CARDS

Cards

[+ Add](#) [✎ Change](#)

TOKEN BLACKLIST

Blacklisted Tokens

[+ Add](#) [✎ Change](#)

Outstanding Tokens

[✎ Change](#)

TRANSACTIONS

Transactions

[+ Add](#) [✎ Change](#)

Recent actions

Authenticated Django administration site.

Credit Card Payment System API 1.0.0

OAS 3.0

[/api/schema/](#)

Django API for authentication, cards, transactions and admin reporting.

Authorize



api



GET

/api/admin/export/



GET

/api/admin/summary/



POST

/api/auth/login/



POST

/api/auth/logout/



GET

/api/auth/me/



POST

/api/auth/refresh/



POST

/api/auth/register/



OpenAPI documentation for the Django REST API.

Credit Card Payment Service 1.0.0

OAS 3.1

[/openapi.json](#)

Simulated payment processing service. No real payment gateway is used.

default

GET	/health	Health	▼
GET	/version	Version	▼
POST	/payments/	Make Payment	▼

Schemas

HTTPValidationError >	Expand all	object
PaymentRequest >	Expand all	object
PaymentResponse >	Expand all	object
ValidationError >	Expand all	object

Payment-service Swagger UI and request schemas.